PA194N6: a different 194-state machine for the consistency of PA

Coefficient-six formula coding, modus ponens without a dummy push, and a current comparison of 47 research packets

Research prepared for Vladimir Reshetnikov

13 November 2026, Pacific time

Abstract

A verified construction below 194 states was not obtained. The requested fallback is supplied by PA194N6, a complete binary, one-tape, blank-start machine with 194 working states and 388 transitions. It combines two previously separate substantive changes: coefficient-six numerical formula constructors and a modus-ponens command that does not push a dummy zero. The first changes every constructor computation; the second changes the stack effect of successful detachment. This is a new composition of supplied ideas, not a claim to have invented either idea separately.

The new table is absent from all 30 recognized 194-state control-graph classes in the pinned corpus, even under arbitrary control renaming, a graph-spanning change of root, reflection, and global symbol exchange. The comparison covers 44 placed research packets and three current incoming archives. It also identifies the purportedly separate PA194X and PA194CSZ machines as byte-identical.

The accompanying ordinary mathematical argument proves the PA-internal consistency equivalence by combining the inherited numerical-calculus calibration with a command compiler valid at coefficient six and a complete physical binding. Two symbolic implementations check all 4,877 primitive source outcomes. The independent implementation derives its register-list lemmas from literal transitions and verifies the entire 58,320-step blank startup. These results do not constitute a new end-to-end Lean or Rocq proof; the distinction and the inherited proof dependencies are explicit.

Contents

1 Result and the exact meaning of the claim **2**

2 Comparison of the complete current corpus **3**

2.1 The 194-state evidence frontier 3

2.2 An exact duplicate and three concrete failed bootstraps 4

2.3 What is materially different about PA194N6 4

3 Literal construction and exact source binding **4**

3.1 The coefficient-six parent and its eight simultaneous folds 5

3.2 The new MP bypass 5

3.3 Constructor semantics 5

4 The coefficient-six numerical calculus and its PA calibration **6**

4.1 The inherited base and the change of numerical coding 6

4.2 Syntactic translation back to PA 7

5 The radix-six no-push command compiler **7**
5.1 Exact numerical operations and command contracts . . . 7
5.2 Uniform stack operations and a one-command discard . . . 8
5.3 Axiom selectors and generalization . . . 9
5.4 Modus ponens with its spare zero supplied by the compiler . . . 9
5.5 Soundness and the PA calibration . . . 10

6 From the source to the literal blank-tape machine **10**
6.1 Proof and verification boundaries . . . 11

7 The search for fewer than 194 states **11**

8 What transfers from ZFC and ConciseIPRA **12**

9 Reproduction, provenance, and deliverable map **12**

A Complete dense transition table **14**

B All 30 pre-existing 194-state graph classes **16**

C All 47 principal packet contributions **18**

1 Result and the exact meaning of the claim

The working-state set is $\{0, \ldots, 193\}$, the initial state is 0, and 194 is the separate terminal halt label. The head starts at cell zero of an all-zero tape indexed by $\mathbb{Z}$. Every defined transition writes one bit and moves exactly one cell left or right. The transition function is total on the 194 working states and the two symbols; no transition is defined at halt. The complete dense table is printed in Appendix A and supplied as `machine/machine.tm`.

Theorem 1 (Ordinary PA-formalizable correctness)**.** *Let M be the literal PA194N6 table in this archive. For a fixed ordinary primitive-recursive proof predicate for first-order Peano arithmetic,*

$$\mathsf{PA} \vdash \mathrm{Halt}(M, \mathrm{blank}) \longleftrightarrow \exists p \, \mathrm{Prf}_{\mathsf{PA}}(p, \ulcorner 0 = 1 \urcorner).$$

Consequently,

$$\boxed{\mathsf{PA} \vdash \neg \, \mathrm{Halt}(M, \mathrm{blank}) \longleftrightarrow \mathrm{Con}(\mathsf{PA}).}$$

The proof uses the supplied ordinary syntactic calibration of the separate-diagonal-head calculus to PA, with its full argument retained and attributed in the archive. It then supplies and checks the new composition's command meaning, source binding, literal implementation, and initialization. The inherited calibration is a mathematical dependency, not a newly added axiom of PA. The theorem is an ordinary mathematical PA-provability claim; this release does not contain a newly kernel-checked end-to-end proof of it. In particular, Python success and a conditional iPRA endpoint are not being presented as such a kernel proof.

The theorem concerns finite coded traces and finite coded proofs, including their interpretation in nonstandard models of PA. It does not rely on observing a long nonhalting run. PA consistency is not an extra hypothesis needed to prove the biconditional. It is the usual additional assumption under which one concludes actual nonhalting in the standard model.

Machine property	Value
Working states / complete binary rows	194 / 388
Dense initial / halt labels	0 / 194
Native initial / halt labels	74 / 254
Source instructions / primitive outcomes	3,776 / 4,877
Unfolded controls / simultaneous folds	202 / 8
Exact blank startup	58,320 transitions

The exact dense-table SHA-256 is

`36da42a81d605ccb8b5edae197c7cde311447bd95b9bf1ab771a3e726a41c6f4`.

The native-table SHA-256 is

`63b6f604ee34ee119eb92ce31250212572041428 54dec416c6b955f0560b975a`.

The achieved state count is still 194. No minimality claim, uniform speed improvement, or 193-state result is asserted. Formula codes at coefficient six can be larger than their coefficient-four counterparts; omission of a push saves work in a particular command but does not imply an improvement in the complete proof search's running time.

2 Comparison of the complete current corpus

All repository claims and source identities are pinned to

`ffbaf45cadbbc98f5dfd0b473cc8d25c9c21fb9b`.

The two requested locations contain 44 placed packet directories under `Computability/BusyBeaver/Research/PA/docs` and three current incoming ZIPs: PA194R, PA194N, and PA194ZR. Together these give 47 principal research packets. A narrative contribution ledger for every packet is included in Appendix C, with exact repository paths in JSON and CSV. Relevant ZFC and ConciseIPRA developments are considered in Section 8.

The fresh recursive scanner examines complete whitespace-separated numeric five-column binary L/R tables in `.tm`, `.txt`, `.tsv`, and `.csv` files. It visits nested ZIPs, deduplicates identical archive payloads, requires two rows per working state, and recognizes a unique undefined halt label. It scanned 18,830 direct files and visited 17 distinct ZIP payloads at 29 archive references without archive parse errors. Source-only, arbitrary JSON, and prose descriptions are outside automatic recognition; the packet ledger covers their reported contributions.

Quantity	Entire recognized corpus	At 194 states
Literal-table occurrences	3,951	283
Distinct byte forms	669	54
Action-graph classes with a spanning root	478	30

Two additional disconnected synthetic examples have six and nine controls; they are not consistency machines and are excluded from the 478 spanning-root classes. There are 16 such classes at 195 states and 99 at 196 states. The counts include ancestors, auxiliary searches, and rejected tables. They are neither counts of established consistency theorems nor state lower bounds. Historical reports used different snapshots, placement policies, and parser scopes, so their raw occurrence counts are not directly comparable.

2.1 The 194-state evidence frontier

The 30 current graph classes divide into 17 classes with supplied ordinary PA arguments and independent physical evidence, nine auxiliary or search classes, and four explicitly rejected implementations. This is an evidence classification of the supplied packets, not a claim to have rerun every old mathematical proof. Appendix B lists all 30.

Family	Contribution and comparison implication
PA194 / PA194-68/8	Same separate-head calculus and controller; different AX decoder allocations, $134 + 67 - 7$ and $134 + 68 - 8$.
PA194-AD	Physical A/D register exchange; the logical source is retained.
PA194C / CS	Cached MP conclusion and cache-based Shift; CS exploits symmetry of a destructive comparison and a contextual fold.
A / F / X / J / ZR	Known-zero tests replace different source jumps; source invariants and blank startup must be checked separately.
PA194R	Reassociated decoder on the F source; changed source addresses, unchanged resolved instruction graph.
M4R / M5 / M6	Reassociated constant-increment decoder; M5 and M6 also change formula coding and need a syntactic proof translation.
PA194D / Q	Respectively move the diagonal head to $\Gamma(D)$ and exchange the curried premises of a quantifier head; each has a new logical interface.
PA194N	Successful MP omits a dummy zero push; the proof-word compiler supplies its own framed zero when it needs one.

2.2 An exact duplicate and three concrete failed bootstraps

The PA194X table in the incoming R packet is byte-identical to PA194CSZ in the placed Q packet. Their common dense-table SHA-256 is

`62e93098caf7c09ff2938d07064f0dbbb47c71ec369892968258fff708592269`.

Both change source site 26 on the symmetric cached controller to a known-zero scratch decrement. Their separate novelty comparisons had used an older eight-class frontier. The present comparison includes both. Several older cached-controller packets also deliver the same PA194C graph.

A separate direct finite-support simulator freshly reproduced blank halting after exactly 1,361 transitions for three distinct rejected comparator-return mutations. Each tries to use a register's clean-source zero invariant during the earlier bootstrap, where that invariant is unavailable. This is a concrete failure, not merely a missing proof. A fourth rejected class has a source-address scanner defect. The existing auxiliary `CS_both_zero6.native.tm` is a different table and must not be confused with those failed comparator mutations.

2.3 What is materially different about PA194N6

The difference is relative to the complete pinned corpus, including its auxiliary tables. Compared with PA194N, the new machine builds implication codes $6\mathsf{p}(a, b)$ rather than $4\mathsf{p}(a, b)$ and correspondingly changes membership, universal, and equality codes. Compared with M6, successful MP retains the old stack instead of pushing a zero. These changes operate in recurring logical computations. They are not merely renaming controls, reflecting the tape, changing unreachable rows, or altering a bootstrap delay.

For a concrete arithmetic difference, $\mathsf{p}(1, 0) = 1$, so the two implication codes are 4 and 6. For a concrete stack difference, at coefficient six consider a successful MP store with $(t, w, d, h) = (0, 1, 0, 12)$ and parameter $x = 1$. Since $I_6(0, 1) = 12$, PA194N6 returns $(1, 1, 1, 0)$. The M6 dummy-push rule returns $(1, 2, 1, 0)$ because $\mathsf{p}(0, 1) = 2$. These are command-interface examples, not claims that a particular inconsistent store occurs in the actual consistent blank run.

The current-corpus comparison finds no match to PA194N6 among all 30 prior 194-state graphs, allowing arbitrary control names, any root that spans the graph, left/right reflection, and global read/write symbol exchange. The last transformation is used only to strengthen the novelty test: the actual machine always starts on zero tape. A finite graph comparison establishes structural distinctness; the mathematical correctness proof is separate.

3 Literal construction and exact source binding

The native labels retain the inherited interpreter's numbering. Native start is 74, native halt is 254, and native source-frame entry is 84. The dense table is the checked breadth-first relabeling from the start, with halt assigned

194. Both formats and their explicit bijection are supplied.

3.1 The coefficient-six parent and its eight simultaneous folds

The M6 parent has 135 fixed controls, including a three-increment reader, and 67 AX readers. Eight compatible history-sensitive folds give

$$135 + 67 - 8 = 194.$$

The coefficient-six multiplier uses the tree $((i, D), (i, D))$, where i is increment of the formula-top register and $D = (i, i)$. All incoming edges to D write one and move right. Another decoder, native 41, is entered only after a written zero. This existing history bit distinguishes the contexts and permits $41 \mapsto 99$ with no new marker.

The added native control 330 has actions $0R61$ and $1R99$; native 190 dispatches both bits to 330, preserving the bit and moving right. The retained control 99 uses $0L30$ and $1L2035$. Four local cases reproduce the old 41/99 effects in three steps, restoring the history cell. Together with the seven inherited AX folds, all 32 local cases are checked on the final simultaneous table. Individual fold correctness is not assumed to imply compatibility automatically.

3.2 The new MP bypass

Starting with M6, change exactly one native transition:

$$(164, 1): \quad 1R98 \quad \longmapsto \quad 1R114.$$

Control 98 enters the push-and-copy offset routine; control 114 enters its copy-offset suffix directly. The physical register arithmetic and AX reader folds are retained. No row in a register arithmetic box changes.

At the source level the change is

$$\texttt{node163} = (\texttt{node6}, \texttt{node55}) \quad \longmapsto \quad (\texttt{node6}, \texttt{node54}),$$

where $\texttt{node55} = (\texttt{node49}, \texttt{node54})$ and node49 is the stack push. The exact M6 graph has 3,795 sites. Its interval $[3756, 3775)$ consists of the 19 removed primitive push instructions. The only incoming edge from outside this interval is $3750 \to 3756$. The new graph has 3,776 sites. The builder checks all retained instructions and all successor indices under the explicit relocation, including bypassing that entry to the retained suffix. It does not infer source identity merely from a macro word's text.

The retained copy suffix has two loops. The first decrements D and increments both T and X; the second decrements X and increments D. From $(T, D, X) = (0, b, 0)$, after $j \le b$ iterations of the first loop the store is $(j, b - j, j)$. After $j \le b$ iterations of the second it is $(b, j, b - j)$. Thus it returns $(b, b, 0)$, preserving the arbitrary stack and all other registers, in $7b + 2$ primitive source steps. These elementary induction invariants explain why the bypass implements exactly the changed MP contract rather than merely making the source shorter.

3.3 Constructor semantics

Registers are ordered $(N, T, W, A, D, H, X, Y, L)$. The constructor macro pops a Cantor stack entry and changes

$$(T, W) = (a, \mathsf{p}(b, w)) \quad \longmapsto \quad (6\mathsf{p}(b, a), w),$$

when entered with $X = Y = 0$; it restores these scratch registers and preserves the others. Its pairing prefix is inherited. Its final loop consumes Y and increments T six times per unit. For arbitrary starting t, y, the invariant

$$T + 6Y = t + 6y$$

and induction on the initial y prove termination with $T = t + 6y, Y = 0$. This supplies the macro meaning beyond the primitive implementation checks.

The same constructor is used in AX and in the MP equality test. A mixed machine with coefficient six in AX and coefficient four in MP would be incorrect. The source binding and free-constructor emitter audit verify that both use the same coefficient. The 61 multiplier bodies of the original source are six increments long; the N bypass deletes a different block and does not change those bodies.

4 The coefficient-six numerical calculus and its PA calibration

Define Cantor pairing and four numerical constructors by

$$\begin{aligned} \mathsf{p}(a,b) &= \frac{(a+b)(a+b+1)}{2} + b, \\ I_k(a,b) &= k\mathsf{p}(a,b), & M_k(a,b) &= k\mathsf{p}(a,b)+1, \\ U_k(i,a) &= k\mathsf{p}(a,i)+2, & E_k(a,b) &= k\mathsf{p}(a,b)+3. \end{aligned}$$

Here M_k in a two-argument expression denotes membership coding, not a Turing machine. The actual machine uses $k = 6$. Zero denotes truth and one the target contradiction. Set $N_k(a) = I_k(a,1)$ and use the classical implicational definitions of conjunction and existential quantification. All variable labels are numerical, and formulas may have free variables.

The calculus $\mathcal{C}_k$ has modus ponens and generalization of theorems, the six inherited PA196 heads, the separated diagonal schema $\Gamma_k(a) = I_k(a, U_k(a,a))$, the guarded separation head, and

$$\rho_k(D,A,H,u) = ((H \to u) \to D) \to ((D \to H) \to (A \to H)), \qquad u \le D.$$

Arrows here mean I_k. The six heads are, with existential quantification read in the coefficient-k constructors,

$$\begin{aligned} h_1(D,H) &= (\exists x_D\, U_k(D,H)) \to H, \\ h_2(D,A,H) &= U_k(D,U_k(A,H)) \to U_k(A,U_k(D,H)), \\ h_3(D,A,H) &= M_k(H,D) \to (E_k(H,A) \to M_k(A,D)), \\ h_4(D,A,H) &= E_k(H,D) \to (M_k(A,H) \to M_k(A,D)), \\ h_5(D,H) &= \exists x_D E_k(H,D), \\ h_6(D,A,H) &= U_k(D,A) \to (U_k(D,A \to H) \to U_k(D,H)). \end{aligned}$$

For the guarded head, put

$$\begin{aligned} P(a,b,c) &= c \to M_k(c,b), \\ V(a,b,c) &= M_k(a,b) \to U_k(c, M_k(a,b)), \\ S(a,b,c) &= U_k\big(b, (M_k(a,b) \to P(a,b,c)) \wedge (M_k(c,b) \to c)\big), \\ Q(a,b,c) &= U_k(c, S(a,b,c) \to N_k(P(a,b,c))), \\ G(a,b,c) &= N_k(V(a,b,c) \to Q(a,b,c)). \end{aligned}$$

Every $G(a,b,c)$ with $c \neq a,b$ is an axiom. A failed emitter guard returns zero. This is legitimate because $\rho_k(0,0,0,0) = 0$ already gives a derivation of zero. Generalization maps a theorem B to $U_k(i,B)$ for any numerical label i, with no undischarged assumptions. This completely specifies the numerical calculus. Its primitive proof translations are given in the retained `calibration/` material. The exact emission order is also verified as free constructor terms, including the saved incoming top and the generalization of the incoming cache. The full calculus is part of the deliverable; it is not an unspecified assertion that the machine checks PA.

4.1 The inherited base and the change of numerical coding

The supplied separated-head calibration provides primitive-recursive translations between PA refutations and $\mathcal{C}_4$ refutations with correctness provable in PA[2, 3]. The diagonal replacement, truth-code alias, and coincident-variable cases are included in that dependency. The following transport to coefficient six addresses two additional issues: variable-label recoding and a numerical order constraint.

For each fixed $k \ge 4$, put $F_k(0) = 0$. For $n > 0$, write uniquely $n = 4\mathsf{p}(a,b) + r$, with $0 \le r < 4$, and define

$$F_k(n) = k\mathsf{p}(F_k(a), F_k(b)) + r.$$

Both components are smaller than n, so primitive-recursive course-of-values evaluation is available. The same function recodes formula numbers and variable labels. An inverse on its image is obtained by the analogous

recursive decoder J_k, returning zero on unused residues $4, \dots, k-1$. Induction proves $J_k(F_k(n)) = n$, hence injectivity. The map fixes 0 and 1 and commutes with all four constructors. In particular,

$$F_k(\Gamma_4(a)) = \Gamma_k(F_k(a)).$$

Keeping old variable labels while recoding formulas would fail at this diagonal schema. Injectivity also preserves the guarded head's inequalities.

The map does not preserve numerical order. For example, $40 < 44$ but $F_5(40) = 75 > 55 = F_5(44)$. Thus a dominated propositional axiom cannot always be recoded as a single dominated axiom. The inherited generic theorem `rho8All` gives a fixed finite derivation of the unrestricted schema from the restricted one[6]. Its order premises hold for I_k because

$$a, b \le \mathsf{p}(a, b) \le I_k(a, b).$$

Replace each problematic axiom leaf by this finite derivation at the recoded parameters. This yields a proof-producing translation

$$\mathsf{PA} \vdash \mathrm{Ref}(\mathsf{PA}) \longrightarrow \mathrm{Ref}(\mathcal{C}_k).$$

The template's own restricted leaves have valid target bounds; no false monotonicity assumption is used.

4.2 Syntactic translation back to PA

Interpret numerical membership as finite binary membership,

$$\mathrm{Bit}(s, z) \equiv \exists u, v\, [s = (2u+1)2^z + v \ \wedge \ v < 2^z],$$

and interpret equality, implication, and universal quantification in their usual first-order meanings. At coefficient six, unused tags 4 and 5 decode as truth. This makes the arithmetic interpretation total; it does not add all unused numbers as axioms of $\mathcal{C}_6$.

Every component and occurring variable label of a nonzero formula code is smaller than that code. Therefore the variable labelled a is absent from the interpretation of formula a, and the diagonal schema has a uniform first-order proof. PA proves $\mathrm{Bit}(s, z) \to z < s$, so code one, $M_k(0, 0)$, translates to the refutable formula $\mathrm{Bit}(x_0, x_0)$.

For each arithmetic formula syntax $\theta(z, \vec{u})$, PA proves finite bit comprehension by induction on n:

$$\forall n\, \exists s < 2^n\, \forall z < n\, (\mathrm{Bit}(s, z) \leftrightarrow \theta(z, \vec{u})).$$

The proof either retains the existing code or adds 2^n according to $\theta(n, \vec{u})$. Substitution into this proof template is primitive recursive on formula syntax. It is not a uniform PA truth predicate or a reflection principle. The retained calibration uses this template for the separation head, handles coincident variable labels separately, and uses capture-avoiding renaming. The remaining heads have fixed logical proof suppliers.

Consequently PA verifies a syntactic translation

$$\mathrm{Ref}(\mathcal{C}_k) \longrightarrow \mathrm{Ref}(\mathsf{PA}).$$

The exceptional equality $I_k(0, 0) = 0$ is handled by a proof of truth equivalent to truth implying truth; numerical stacks and formulas are not treated as free syntax in this case. This completes the inherited radix calibration needed at $k = 6$[4]. The present construction's additional obligation is to compile refutations into its changed command language.

5 The radix-six no-push command compiler

5.1 Exact numerical operations and command contracts

Let

$$p(a, b) = \frac{(a+b)(a+b+1)}{2} + b, \qquad [a; w] = p(a, w), \qquad I(a, b) = 6p(a, b), \qquad U(i, a) = 6p(a, i) + 2.$$

Iterated brackets put their leftmost entry at the top. The Cantor projections are total and satisfy

$$\mathrm{hd}[a;w]=a,\quad \mathrm{tl}[a;w]=w,\quad [0;0]=0,\quad \mathrm{hd}(0)=\mathrm{tl}(0)=0.$$

A numerical stack is therefore not being treated as a free list. A command store is (t,w,d,h): current formula, stack, offset, and cached formula. Code zero is truth and code one is the target contradiction. The new MP command, with $y=d+x$, is

$$\mathsf{MP}_x(t,w,d,h)=\begin{cases}(y,w,y,0), & I(t,y)=h,\\ (0,w,y,0), & I(t,y)\neq h.\end{cases} \tag{1}$$

Shift is retained literally:

$$S(t,w,d,h)=\begin{cases}\mathsf{halt}, & t+h=1,\\ (h,\mathrm{tl}(w),d,\mathrm{hd}(w)), & t+h\neq 1.\end{cases} \tag{2}$$

For $D=d+x$, $H=h+y$, the unchanged AX command has output

$$\mathsf{AX}_{A,x,y}(t,w,d,h)=\big(g(H,D,A),[q_1,\dots,q_9,t;w],0,0\big). \tag{3}$$

Here $q_1=\Gamma(A)=I(A,U(A,A))$, $q_2,\dots,q_7$ are the six retained head schemas in reverse emission order, interpreted with the radix-six constructors defined in Section 4,

$$q_8=\rho(D,A,H,d)=I(I(I(H,d),D),I(I(D,H),I(A,H))),\qquad q_9=U(d,h).$$

The gate g is the separate-head guarded separation gate at coefficient six. The inherited free-constructor emitter audit is retained in `construction/inputs/parent6.emitter_audit.json`. The new source splice occurs wholly after that emitter and retains all of its resolved instructions and successor addresses under relocation. All $q_1,\dots,q_8,g$ are divisible by six, and $q_9\equiv 2 \pmod 6$, with $q_9\geq 2$. In particular none is code one. This is a statement about constructor values, requiring no consistency assumption and holding for arbitrary incoming natural stores.

The count in (3) is nine generated stack entries plus the saved old top. The two identities needed below are

$$\mathsf{AX}_{A,x,y};S^9(t,w,d,h)=(q_8,[t;w],0,U(d,h)), \tag{4}$$

$$\mathsf{AX}_{A,x,y};S^{10}(t,w,d,h)=(U(d,h),w,0,t). \tag{5}$$

All displayed Shifts are nonhalting: their tests involve two generated non-one codes, zero and a generated code, or the universal code of value at least two. This also establishes exact stack effects when zero pairs collapse.

5.2 Uniform stack operations and a one-command discard

Write $A_0=\mathsf{AX}_{0,0,0}$ and define

$$\mathsf{Push}=A_0;S^9;\mathsf{MP}_0,\qquad \mathsf{Pop}=A_0;S^{11},\qquad \mathsf{Discard}=\mathsf{MP}_0,\qquad \mathsf{Offset}_i=\mathsf{MP}_i.$$

Their exact contracts are

$$\mathsf{Push}(t,w,d,h)=(0,[t;w],0,0), \tag{6}$$

$$\mathsf{Pop}(t,[a;w],d,h)=(t,w,0,a), \tag{7}$$

$$\mathsf{Discard}(t,w,0,h)=(0,w,0,0), \tag{8}$$

$$\mathsf{Offset}_i(0,w,0,0)=(0,w,i,0). \tag{9}$$

For Push, (4) makes the final MP compare an implication with a universal code, so it fails with offset zero. For Pop, the first ten Shifts give (5). The eleventh Shift tests $U(d,h)+t\geq 2$, hence it does not halt even when $t=1$. It pops exactly the displayed item. Thus Pop is total on all natural stores. In particular $\mathsf{Pop}(t,0,d,h)=(t,0,0,0)$ and $\mathsf{Push}(0,0,d,h)=(0,0,0,0)$.

For Discard, the two MP outcomes in (1) coincide when $d=x=0$, for arbitrary h: both are $(0,w,0,0)$. This gives a single-command discard, where the old dummy-push controller needed a longer word. The old cache need not be zero for this identity. For Offset, if $i>0$ then $I(0,i)>0$, so the comparison fails; if $i=0$ it succeeds and returns $(0,w,0,0)$. The same one-command definition covers both cases. These simplifications affect the proof-word compiler; they do not claim additional physical state savings.

5.3 Axiom selectors and generalization

In the following contracts the arbitrary incoming store is $(t, w, 0, 0)$. For $1 \le j \le 7$, the word

$$\mathsf{Discard}; \mathsf{AX}_{A,D,H}; S^{j+1}; \mathsf{Pop}^{9-j}$$

returns $(q_j, w, 0, 0)$ with the parameters evaluated at the desired A, D, H. After the initial Shifts, the remaining $9 - j$ stack entries are exactly the unselected generated tail and the saved zero. The Pops preserve the selected top and consume those entries; the last pop puts that saved zero into the cache. For the dominated head, if $u \le a$, use

$$\mathsf{Discard}; \mathsf{Offset}_u; \mathsf{AX}_{b,a-u,c}; S^9; \mathsf{Pop}.$$

It returns $(\rho(a, b, c, u), w, 0, 0)$, with the numerical dominance condition coming from $a = u + (a - u)$. A permitted gate is obtained by

$$\mathsf{Discard}; \mathsf{AX}_{A,D,H}; \mathsf{Pop}^{10},$$

which returns $(g(H, D, A), w, 0, 0)$. Truth uses Discard alone.

The word

$$\mathsf{Gen}_i = \mathsf{Push}; \mathsf{Offset}_i; S; \mathsf{AX}_{i,0,0}; S^{10}$$

returns $(U(i, a), w, 0, 0)$ from $(a, w, 0, 0)$. Before its AX the store is $(0, w, i, a)$; its last ten Shifts use (5) with saved top zero. Its first Shift tests $0 + 0$, and the later ones are among the safe AX shifts. Hence this generalization word is total even for $a = 1$.

5.4 Modus ponens with its spare zero supplied by the compiler

Suppose recursively compiled proofs return a and $I(a, b)$ with the stack framed and both caches cleared, or halt earlier. The MP compiler is

$$\mathsf{Discard}; \mathsf{Push}; C(I(a, b)); \mathsf{Push}; C(a); \mathsf{Push}; S; S; \mathsf{MP}_b; \mathsf{Pop}. \tag{10}$$

The first two commands reserve a zero below the premise computations. If the recursive calls have returned, the store before the two Shifts is

$$(0, [a, I(a, b), 0; w], 0, 0).$$

The first Shift returns $(0, [I(a, b), 0; w], 0, a)$. The second halts if $a = 1$; otherwise it returns $(a, [0; w], 0, I(a, b))$. MP succeeds and returns

$$(b, [0; w], b, 0).$$

By the total Pop contract, the final word returns $(b, w, 0, 0)$, including $b = 1$. The stack zero is now consumed once, at the compiler level. At $a = b = w = 0$ the same proof uses the literal Cantor projection identities; no extra distinction between a zero push and an empty stack is introduced.

Induction on a finite derivation p in the inherited separate-head calculus therefore proves:

> From every $(t, w, 0, 0)$, $C(p)$ either halts or returns $(\mathrm{concl}(p), w, 0, 0)$.

The axiom and generalization cases use the words above. The MP case is (10); extending a word after an early halt still halts. In particular a refutation compiler ends in top one, if it has not already halted. Its final appended Shift halts because $1 + 0 = 1$. For arbitrary incoming t, w, d, h, first apply A_0 to clear d, h. Thus the concrete word

$$A_0; C(p); S$$

halts from *every* incoming store whenever p derives code one. The compiler is a primitive-recursive traversal of the finite derivation, and PA proves its correctness by induction on that coded derivation. It does not assume a reset between the scheduler's unsuccessful trials.

The unchanged scheduler enumerates finite command words with the usual Cantor continuation coding. A cofinal family of halting fresh candidates is obtained by prefixing the universal word by $\mathsf{AX}_{k,0,0}$. Let $E(\epsilon) = 0$ and let a positive continuation be encoded by $E(c :: w) = 1 + \mathrm{encodeCommand}(c, E(w))$. In particular

encodeCommand($\mathsf{AX}_{A,x,y}, z) = p(A+2, p(y, p(x, z)))$. Put $z = E(A_0; C(p); S)$, using this positive continuation convention rather than a fresh-candidate code. The fresh candidate codes for the prefixed words are exactly $p(k+2, p(0, p(0, z)))$, which are strictly increasing and unbounded in k. The fresh counter begins at two, so omitting smaller candidates loses none of this cofinal family. Every decoded continuation strictly decreases, and the unchanged source macro loops terminate. The resulting finite-word and finite-trace construction is formalizable in PA by primitive recursion and induction.

5.5 Soundness and the PA calibration

Annotate the current top t, cached head h, and every live canonical Cantor-stack entry by a derivation in the separate-head calculus, or by a PA proof of its arithmetic translation. For $w > 0$, its decoded tail is strictly smaller than w; thus the canonical stack and its annotation have a primitive-recursive bounded construction. A zero stack supplies truth when popped, and pushing zero onto zero preserves the empty canonical annotation.

AX adds the unchanged valid axiom instances, a generalization of the old cached theorem, and the saved old top. On a successful MP, the annotations of t and $I(t, y) = h$ produce an annotation for y. The stack is retained, and the cache receives the truth proof. Failure outputs truth and retains the stack. Shift transports the annotations. If its test succeeds, then $t + h = 1$ implies that one of the two annotated formulas is code one. The inherited syntactic arithmetic translation turns that annotation into a PA refutation. No annotation is imposed on the arithmetic offset d.

Consequently PA proves both finite syntactic reductions:

$$\text{source halt} \longrightarrow \text{PA refutation}, \qquad \text{PA refutation} \longrightarrow \text{source halt}.$$

For the second reduction, first use the supplied PA-to-radix-four translation and then the hereditary recoding and restricted-axiom proof templates of Section 4 to obtain a radix-six refutation. Apply the all-store compiler above to that refutation. For the first, compose the proof annotation with the radix-six-to-PA translation of that same section. This uses the existing logical calibration, including its treatment of the code-zero alias and coincident variable labels. Nothing in the no-push modification changes that calculus or requires a new logical axiom, a reflection principle, or consistency as an additional assumption.

6 From the source to the literal blank-tape machine

A source frame consists of the encoded source address, a zero gap with header parameter 48, nine arbitrary natural register values encoded in unary blocks, and an arbitrary finite nonempty spare list. For a source address p, the gap length is $48 + n - |p|$ for arbitrary $n \in \mathbb{N}$. The contracts quantify over these data, not over a bounded sample of small register values. Both outcomes of every decrement are checked, even when a particular program invariant would rule one out from the seed.

The construction executor verifies 4,877 primitive outcomes for the 3,776-site source. A separate implementation resolves the source graph in two ways, checks native/dense conjugacy, and derives four generic list-sweep lemmas from the actual table rows. Its primitive traces import neither the builder nor an arithmetic-box oracle. Each nonhalting contract ends at the exact complete successor frame without passing through halt, and its duration has a strictly positive constant term.

Verified quantity	Construction executor	Independent executor
Source instructions	3,776	3,776
Primitive outcomes	4,877	4,877
Symbolic operations	1,531,094	1,115,525
Maximum operations per outcome	499	397
Minimum positive duration constant	—	3
List lemmas derived from literal rows	inherited boxes	4
Exact startup transitions	58,320	58,320

The operation counts describe certificate execution, not a uniform number of Turing-machine transitions for arbitrary register values. A third review separately replays all 32 fold traces and all 28 incoming-context rows,

checks the source splice and label bijection, and simulates startup in lockstep with M6. It confirms that the edited row is never read during startup or inside a fold trace.

The bootstrap is checked independently of all clean-frame invariants. After exactly 58,320 transitions the blank machine is at native control 84 with head -2, source site 25, logical registers $(2, 0, 0, 0, 0, 0, 0, 0, 0)$, and 26 spare blocks each containing one. The receipt records every nonzero tape cell. Equality includes the zero exterior, not just a finite window.

Lemma 2 (PA-internal physical transport)**.** *PA proves that the literal PA194N6 machine halts from blank if and only if its source program halts from this clean seed.*

Proof. The fixed startup and each primitive contract are finite row calculations and unary-list inductions. PA formalizes these calculations, their frame relations, and concatenation of coded traces. A finite source halt gives a finite literal halt by concatenating the positive primitive segments.

For the reverse implication, suppose a literal halt occurs by time T. A halt during startup contradicts its checked trace and determinism. Otherwise, if the source has no halt in its first $T + 1$ instructions, the total increment, decrement, and jump operations give $T + 1$ completed physical segments after startup. Every segment has positive duration and no intermediate halt, so this deterministic physical run continues beyond T, a contradiction. Thus a bounded finite source halt exists.

At source time m every register is bounded by its initial value plus m, since each primitive increases at most one register by one. Coded source iteration is primitive recursive. Physical list-sweep proofs and their finite compositions use induction on natural lengths. This is a proof about arithmetized finite traces in PA, rather than an inference from standard-model simulation alone. □

The all-store proof-word compiler of Section 5, the calculus translations of Section 4, and this physical transport yield the theorem in Section 1.

6.1 Proof and verification boundaries

The fresh facts are the literal table identity, one-row M6 patch, exact source splice, generalized no-push command compilation, current-corpus distinctness, two complete symbolic primitive checks, and exact startup. The finite certificates rely on the mathematical soundness of their symbolic rules; Python execution is not a kernel proof. The inherited base calibration is an ordinary mathematical dependency whose frozen argument and source interfaces are retained.

The current repository explicitly says that both machine-specific iPRA directions remain open for the principal PA194, PA195, and PA196 arithmetic endpoints[1]. The imported PA196 formal theorem gives the PA-refutation-to-halting direction, equivalently nonhalting implies PA consistency; it alone does not establish the converse internal implication. Neither that one-direction theorem nor metatheoretic nonhalting is being substituted for the missing direction. No new end-to-end Lean or Rocq proof or iPRA theorem is claimed here.

7 The search for fewer than 194 states

Two new grammar moves were explored beyond single local rotations. A global dynamic-programming reparser rebuilds the 190-token AX word using distinct substrings and randomized repetition costs. A second reparser replaces a whole subtree using the currently shared subtree dictionary. Both preserve the macro word and use the inherited simultaneous-fold scorer and scanner penalties; any promising score would still need exact source and physical validation.

Search	Proposal slots	Time cap	Observed result
Global word reparse	3,278	180 s	Best 60 effective AX controls; total 194.
Dictionary subtree reparse	1,113,967	180 s	Best total 194; 733,412 accepted slots.
MILP-skeleton binarization	1,220,188	60 s	Best penalized reader score 69; no accepted machine.

Proposal slots include repetitions, no-ops, and skipped candidates; they are not counts of distinct verified machines. CPU-time caps are dependent on the execution environment.

A separate mixed-integer model considers the 226 repeated AX substrings and the whole word, minimizing the sum of right-hand-side lengths minus one, with inherited increment macros free. After 180 seconds HiGHS had feasible cost 67 and reported a dual bound 63, with a remaining gap. This is a finite grammar objective, not a physical state objective; the numerical solver output is not a formally checked lower-bound certificate.

The new PA194N6 table also received fresh local-state screens. Collecting all 4,877 source outcomes, generic list proofs, unary-loop interiors, and the complete startup produced 19,318 symbolic local views. Their conservative radius-five expansions give 29,282 eleven-cell windows covering every one of the 194 controls. With a 300-step local bound, a common-action merger and a per-incoming-edge bypass each returned zero candidates. A separate uniform-history screen of the unfolded 202-control parent considered all 129 eligible controls and found exactly the eight existing disjoint fold opportunities, with no ninth.

The complete drivers, frozen inputs, receipts, and empty candidate lists are under `research/`. The new searches were completed; historical million-proposal counts are not being passed off as fresh work. These negative results leave open larger local windows, longer detours, simultaneous changes to several helper controls, new tape encodings, different logical interfaces, and other global redesigns. They give no lower bound for arbitrary PA-consistency machines. The accepted fallback does not depend on the success or reproducibility of a stochastic search.

8 What transfers from ZFC and ConciseIPRA

The ZFC lineage supplies useful engineering methods: semantic command changes with explicit proof compilers; sharing control states through already-written tape bits; separate accounting of source subtrees, physical readers, and contextual folds; and whole-tape initialization checks. Its omit-zero variants also show that the no-push principle itself predates this packet[8]. The new work here is the exact composition at 194 states and its bindings, not a new claim about the general principle.

The ZFC results must retain their theory and formalization bases. A machine for Con(ZFC) cannot automatically be substituted for one for Con(PA) merely because it uses an attractive controller. The current ZFC README also distinguishes externally compiled K215 equivalences from the separate internal PA endpoint for a 240-state machine. State reductions and logical-strength calibrations are different obligations.

ConciseIPRA demonstrates that a small native calculus can derive much of the familiar logical infrastructure from a few computational and closure rules[9]. Its proof-checker compression is a useful source of ideas for finite certificate design and future formalization. Its checker size, however, is not a binary-TM state count. Replacing the searched theory by iPRA would also require a separately proved consistency-strength bridge to PA; that bridge is not supplied just by a compact presentation of iPRA. The present machine therefore retains the calibrated PA-strength numerical calculus.

9 Reproduction, provenance, and deliverable map

Python 3.10 or later and its standard library suffice for the accepted construction and its independent physical audit. Run with assertions enabled, without `-O` or `PYTHONOPTIMIZE`. From the package root:

```
python3 -B construction/build.py --output generated
python3 -B verification/check.py generated
python3 -B verification/review_structure.py
python3 -B comparison/check_current_novelty.py generated/machine.tm
```

The first two commands reconstruct the literal table, complete source, folds and certificates, then run the separate literal implementation. The third is a read-only structural/fold/startup audit against the frozen parent. The fourth compares the new table with the bundled current census. No network or repository checkout is required for these commands.

Optional local reduction screens use a C++17 compiler:

```
cd research
python3 replay_physical.py
```

The research README gives the longer grammar-search commands; only the optional MILP experiment requires NumPy and SciPy. Recomputing the full external corpus inventory additionally requires a checkout of the pinned ProveIt revision and `rg`; this is separate from verifying the machine.

To regenerate the report's data tables and compile its PDF:

```
cd report
python3 generate_includes.py
pdflatex -interaction=nonstopmode -halt-on-error pa194n6_research.tex
pdflatex -interaction=nonstopmode -halt-on-error pa194n6_research.tex
```

Run one further LaTeX pass if requested for references or long-table widths. Standard TeX Live packages named in the preamble are sufficient.

The new complete machine definition is included directly in this report, so reproducing a machine does not depend on the continued availability of a repository URL. The archive includes the LaTeX source and all included fragments, compiled PDF, machine tables and label map, source descriptions, standalone construction and independent verification code, recorded receipts, frozen mathematical dependencies and their licenses, current comparison data, and bounded-search evidence. The checksum manifest binds every delivered file other than itself. Rebuilding elapsed-time receipts may change their hashes; the transition-table identities are fixed.

References

[1] ProveIt, *PA consistency machines*, status at `ffbaf45cadbb`. Pinned README.

[2] ProveIt, *A 194-state Turing machine calibrated to Con(PA)*, primary construction and separate-head source. Pinned packet.

[3] Supplied PA195/PA194 diagonal and separate-head calibration arguments, retained with attribution under `calibration/` in this archive. See the primary packet and its exact provenance files.

[4] ProveIt, *New 194-state PA-consistency machines by changing the constructor coefficient*, M4R, M5, and M6; generic radix calibration. Pinned packet.

[5] ProveIt, *PA194N*, no-push MP construction and its ten-entry command compiler, current incoming archive. Pinned archive.

[6] James Chang, imported `M6/Logic/RestrictedImplication.lean`, generic `rho8All`. Pinned source.

[7] James Chang, imported PA196 source, logical interfaces, and physical interpreter. Pinned snapshot.

[8] ProveIt, ZFC consistency-machine research. Pinned research tree.

[9] ProveIt, *Concise presentations of intuitionistic primitive recursive arithmetic*. Pinned project.

A Complete dense transition table

The entries are $w\,\mathrm{D}\,q'$: write w, move in direction D, and enter state q'. The initial state is 0; halt is 194 with no outgoing rows. All 194 working states are listed. State names are part of this literal presentation, not additional instructions on the tape.

q	Read 0	Read 1	q	Read 0	Read 1
0	1 R 1	1 R 2	97	0 R 117	1 R 97
1	1 R 3	1 R 1	98	0 R 30	1 R 49
2	0 R 0	1 R 2	99	0 R 48	1 R 61
3	0 R 4	0 R 1	100	0 R 87	1 L 118
4	1 R 5	1 L 6	101	0 R 106	0 L 14
5	1 L 7	1 L 8	102	0 R 36	1 R 119
6	0 L 7	1 L 6	103	0 R 118	1 R 79
7	0 L 9	1 L 6	104	0 L 46	0 R 120
8	0 L 10	0 L 11	105	0 L 45	1 R 121
9	1 R 12	0 L 9	106	0 L 96	1 R 122
10	0 L 9	0 L 10	107	0 R 123	0 L 14
11	1 L 8	1 L 11	108	0 R 72	1 R 108
12	0 R 12	0 R 13	109	0 R 120	1 R 109
13	0 L 14	0 R 13	110	0 L 46	0 R 108
14	1 L 15	0 L 15	111	0 R 71	0 R 97
15	0 L 15	1 L 16	112	0 R 59	0 R 97
16	0 L 17	1 L 16	113	0 L 65	1 R 124
17	0 L 18	1 L 16	114	0 R 125	1 R 126
18	0 L 19	1 L 16	115	0 R 109	1 R 115
19	0 L 20	1 L 16	116	0 L 127	1 R 128
20	0 L 21	1 L 16	117	0 R 115	1 R 117
21	0 L 22	1 L 16	118	0 R 129	1 L 67
22	0 L 23	1 L 16	119	0 R 130	1 R 131
23	0 R 23	1 R 24	120	0 R 14	1 R 120
24	0 R 25	1 R 26	121	0 R 94	0 L 14
25	0 R 27	1 R 28	122	0 R 96	1 L 100
26	0 R 29	1 R 30	123	0 R 132	1 R 132
27	0 R 31	1 R 194	124	0 R 133	1 R 134
28	0 L 32	1 R 33	125	0 R 32	1 R 36
29	0 R 34	1 R 35	126	0 R 31	1 R 103
30	0 L 36	1 L 37	127	0 R 135	1 R 136
31	0 R 38	1 R 39	128	0 R 137	1 R 138
32	0 R 40	1 R 41	129	0 R 59	0 R 117
33	0 R 42	1 R 43	130	0 R 104	1 R 121
34	0 R 44	1 R 45	131	0 R 83	1 R 82
35	0 R 46	1 R 47	132	0 R 94	1 R 48
36	0 R 48	1 R 49	133	0 R 138	1 R 139
37	0 R 50	1 R 51	134	0 R 140	1 R 141
38	0 R 52	1 R 42	135	0 R 142	1 R 94
39	0 R 53	1 R 54	136	0 L 134	1 R 143
40	0 R 55	1 R 56	137	0 R 41	1 L 144
41	0 R 57	1 R 58	138	0 R 145	1 R 146
42	0 R 59	0 R 60	139	0 R 147	1 R 148
43	0 R 61	0 L 14	140	0 R 149	1 R 69
44	0 R 62	1 R 63	141	0 R 150	1 R 151
45	0 R 55	1 R 64	142	0 R 152	1 R 153
46	0 L 65	1 R 66	143	0 R 146	1 R 138
47	0 R 36	1 R 67	144	0 R 154	1 R 146
48	0 L 68	1 L 69	145	0 R 102	1 R 67
49	0 R 70	0 L 14	146	0 R 155	1 R 48

Continued on the next page

q	Read 0	Read 1	q	Read 0	Read 1
50	0 R 71	0 R 72	147	0 R 156	1 R 157
51	0 R 73	0 L 49	148	0 R 136	1 R 158
52	0 R 74	1 R 54	149	0 R 145	1 R 159
53	0 L 75	0 L 15	150	0 R 152	1 L 139
54	0 R 76	0 L 10	151	0 R 160	1 R 161
55	0 R 77	1 R 78	152	0 R 102	1 R 162
56	0 R 79	1 R 80	153	0 L 138	1 R 94
57	0 R 81	1 R 82	154	0 R 153	1 R 41
58	0 R 83	1 R 84	155	0 R 80	1 R 41
59	0 L 85	0 R 1	156	0 R 28	1 R 163
60	0 R 86	1 R 60	157	0 L 164	1 L 165
61	0 L 77	1 L 68	158	0 R 166	1 R 167
62	0 R 87	0 L 7	159	0 R 152	1 R 137
63	0 R 88	1 R 89	160	0 R 168	1 R 169
64	0 R 42	1 R 90	161	0 R 170	0 L 10
65	0 R 91	1 R 49	162	0 R 91	1 R 64
66	0 R 92	0 L 76	163	0 R 171	1 R 172
67	0 R 68	1 R 33	164	0 R 173	1 R 167
68	0 R 74	1 R 93	165	0 R 174	1 R 166
69	0 R 94	1 R 95	166	0 R 159	1 R 41
70	0 L 96	1 L 33	167	0 R 145	1 R 153
71	0 L 85	0 R 2	168	0 R 175	1 R 127
72	0 R 97	1 R 72	169	0 R 176	1 R 177
73	0 R 76	1 R 98	170	0 R 178	1 R 179
74	0 R 59	0 R 86	171	0 R 180	1 R 181
75	0 R 68	1 R 99	172	0 L 182	1 L 183
76	0 R 42	0 L 13	173	0 R 152	1 R 150
77	0 R 87	1 R 37	174	0 R 172	1 R 184
78	0 R 30	1 R 100	175	0 R 69	1 R 172
79	0 R 42	1 R 101	176	0 R 137	1 R 185
80	0 R 102	1 R 103	177	0 R 186	1 R 91
81	0 R 104	1 L 105	178	0 R 31	1 R 53
82	0 R 30	1 R 81	179	0 R 187	1 R 178
83	0 R 99	1 R 106	180	0 R 145	1 R 155
84	0 R 30	1 R 107	181	0 R 32	1 R 188
85	1 R 85	1 L 15	182	0 R 157	1 R 189
86	0 R 108	1 R 86	183	0 R 190	1 R 191
87	0 L 46	0 R 109	184	0 R 41	1 R 192
88	0 R 110	1 R 100	185	0 R 164	1 R 158
89	0 R 110	1 R 70	186	0 R 172	1 R 183
90	0 R 111	0 L 14	187	0 R 98	1 R 118
91	0 R 112	1 L 75	188	0 R 193	1 R 116
92	0 R 113	1 R 114	189	0 R 144	1 R 165
93	0 R 94	1 R 30	190	0 R 137	1 R 142
94	0 R 71	0 R 115	191	0 R 182	1 R 154
95	0 R 116	1 R 94	192	0 R 154	1 R 41
96	0 R 50	1 R 105	193	0 R 80	1 R 157

B All 30 pre-existing 194-state graph classes

"Supported" records supplied ordinary PA arguments and independent physical evidence; it does not mean a newly checked proof-assistant endpoint. "Auxiliary" marks search or supplementary objects without assigning a complete new calibration proof. "Rejected" marks a documented physical failure. Exact paths, hashes, and all recognized occurrences are in the machine-readable frontier and census.

Class	Status	Distinguishing construction
Cached I-tree zero-jump seed	Auxiliary	Alternate decoder plus cached controller and entry zero jump; search basis.
CS both-zero6	Auxiliary	Known-zero X decrement at both source26 and final emitter exit; research composition.
CSZ-A3	Auxiliary	CSZ plus comparator-return register3 and final-emitter register3 decrements.
CSZ-A5	Auxiliary	CSZ plus comparator-return register3 and final-emitter register5 decrements.
CSZ-A6	Auxiliary	CSZ plus comparator-return register3 and final-emitter register6 decrements.
Gamma(D)-CSZA	Auxiliary	Supplementary Gamma(D) emitter plus known-zero changes.
M4R	Supported	Reassociated coefficient4 constructor decoder; original resolved source graph.
M5	Supported	Formula constructor coefficient5; explicit variable and proof recoding.
M6	Supported	Formula constructor coefficient6; explicit variable and proof recoding.
MPH	Auxiliary	MP result cached in H while retaining original Shift; physical search input.
PA194	Supported	Separate diagonal head, 67-reader decoder with seven folds, original controller.
PA194-68/8	Supported	Same separate-head source, reassociated 68-reader decoder with eight folds.
PA194-AD	Supported	PA194 with physical A/D register exchange.
PA194A	Supported	CS entry jump at source26 becomes decrement of known-zero register A.
PA194altC	Auxiliary	Cached controller on alternate 68-reader decoder.
PA194C	Supported	MP result remains cached in H; Shift tests H=1; original ten-entry logical interface recompiled.
PA194CS	Supported	Symmetric Shift guard; cached MP controller; contextual controller fold.
PA194CSZ = PA194X	Supported	CS entry jump at source26 becomes decrement X; two headline names denote identical dense bytes.
PA194CZ	Auxiliary	Cached controller; entry known-zero decrement of register 6.
PA194D	Supported	Separate diagonal axiom uses Gamma(D), new proof selector, 69 readers and nine folds.
PA194F	Supported	CS final emitter jump becomes decrement of known-zero register X.
PA194J	Supported	Ten source jumps become known-zero decrements; three physical rows change.
PA194N	Supported	Successful MP drops dummy-zero push, retains original Shift; revised command proof compiler.
PA194Q	Supported	Quantifier-distribution curried premises exchanged; K/S proof translation.
PA194R	Supported	Reassociated decoder of PA194F; 1111 source paths changed, same resolved source graph.
PA194ZR	Supported	Eight comparator returns and final emitter return become known-zero register3 decrements.

Continued on the next page

Class	Status	Distinguishing construction
Rejected CS node1-X	Rejected	Native103/read1 becomes dec6; exact blank run halts at1361.
Rejected I-tree node1-X	Rejected	Known-zero source rewrite fails blank bootstrap; exact halt at1361.
Rejected seven-zero scanner	Rejected	Decoder local folds pass, but source-address scanner violates runtime contract.
Rejected ZR node1-X	Rejected	Register6 comparator-return variant; exact blank run halts at1361.

C All 47 principal packet contributions

The ledger records the reported contribution of every placed or incoming principal packet at the pinned commit. The early packets' historical claims and searches are credited, not silently upgraded to new proofs. Incoming archive hashes and exact locations are preserved in JSON.

Packet	Reported contribution
1. `pa_unary_power_cover`	Unary power covering at PA strength (report of 27 October 2026). The U208 packets carry it as a baseline.
2. `U208_ConPA_Conditional_Context_2026-10-29`	U208 by a conditional tape-context reduction. States a 208-state candidate 'machine/u208.tm'.
3. `U208_ConPA_Entry_Context_2026-10-29`	U208 by eliminating a state using entry context. States a 208-state machine 'machine/u208.tm'.
4. `U208_ConPA_History_2026-10-29`	U208 by a finite-history decoder fold. States a 208-state candidate 'machine/u208.tm'.
5. `U208_ConPA_2026-10-29`	U208 by a three-cell fold. States a 208-state candidate 'machine/u208.tm'.
6. `u208_research_2026-10-29`	U208 research release. States two 208-state machines, 'machine/u208.tm' and 'machine/variant_plain/u208.tm'.
7. `Below_U208_Research_2026-10-29`	Research toward fewer than 208 states. States that no machine with 207 states or fewer was obtained.
8. `u208_comparison_research_2026-10-29`	Comparison of the 208-state proposals and a search below 208. States that the target was not achieved.
9. `ConPA_below208_research_2026-10-29`	Comparative audit and reduction search below 208. States that it does not establish a 207-state machine.
10. `pa_consistency_research_2026-10-29`	Audit and new source reductions. States that no validated machine below 208 states was obtained.
11. `pa208_research_2026-10-29`	Comparison and research below 208 states. States that it does not establish a valid machine below 208.
12. `pa_tm_research_2026-11-13`	Comparison and reduction research. States a new 196-state variant with shorter source paths and nothing below 196.
13. `pa_consistency_research_2026-11-13`	Comparison and diagonal-vacuity research. States a shorter axiom program with a 199-state implementation and nothing below 196.
14. `conpa_tm_comparison_and_reduction_search`	Comparison and reduction search. States two experimental 196-state command variants and nothing below 196.
15. `pa196_research_2026-11-13`	PA196 comparison, source factorization and proof obligations. States a source tree of 157 internal nodes and nothing below 196.
16. `pa_consistency_research`	Research and verification packet. States that no machine below 196 states has been validated.
17. `pa_tm_comparison_and_reductions`	Comparison and new rewrites, a research snapshot.
18. `pa_tm_extended_research_2026-11-13`	Comparison and refined reduction search. States that the smallest accepted count is 196.
19. `conpa_tm_comparison_and_search`	Comparison and a search below 196. States that none was obtained; files named '195' are rejected or diagnostic candidates.
20. `pa_consistency_reduction`	Comparison and reduction experiments. States two new 196-state variants and a shorter source program with a 203-state implementation.
21. `pa_consistency_comparison_2026_11_13`	Comparison and new reduction experiments. States that nothing below 196 was obtained.
22. `PA_consistency_TM_research_2026-11-13`	Comparison and new source reductions. States that no machine below 196 is claimed.
23. `pa_consistency_research_2026-11-13_2`	Comparison and search below 196. States two new layouts with 196 and 198 states and a command-level reduction.
24. `conpa_tm_study_2026-11-13`	Comparison and reduction study. States that nothing below 196 was obtained.

Continued on the next page

Packet	Reported contribution
25. pa_below196_research	Comparison and reduction experiments. States that nothing below 196 was obtained.
26. pa_consistency_comparison_and_new196_2026-11-13	Comparison and reduction research. States that it contains no accepted machine below 196.
27. pa_tm_research_2026-11-13_2	Comparison and reduction investigation. States same-size work, failed candidates and nothing below 196.
28. pa_consistency_cross_archive	Cross-archive constructions and reduction search. States that it does not certify a 195-state machine.
29. pa195_conpa_2026-11-13	PA195. States a 195-state machine ‘pa195/pa195.tm‘ and an article with a PA proof of the equivalence.
30. pa_consistency_reduction_audit_2026-11-14	Comparison and reduction audit. States nothing below 196 and no new theorem.
31. pa_consistency_research_2026-11-14	Comparison and NEG18. States an 18-instruction encoded negation with a compiled machine of 197 states.
32. PA195_ConPA_research_2026-11-14	A 195-state machine. States the primary ‘machine_195S.tm‘ and a second table ‘machine_195C.tm‘.
33. pa195_followup_research	Comparison and searches below 195. States a graph-distinct 195-state machine M195S8 and nothing below 195.
34. pa194_conpa_2026-11-13	A 194-state machine. States the table ‘construction/tables/pa194.tm‘ and a proof of the equivalence in PA.
35. pa194_conpa_2026-11-13_2	PA194. States a 194-state machine ‘pa194/pa194.tm‘ and an article with a PA proof of the equivalence.
36. pa195_diagonal_combined_2026-11-13	PA195DC. States a different 195-state machine.
37. pa194C_conpa_2026-11-13_2	PA194C. States a materially different 194-state machine, an article with a PA proof of the equivalence, and nothing below 194.
38. pa194_new_designs_2026-11-13	Two new 194-state machines, PA194C and PA194CS. States that nothing below 194 was obtained.
39. pa194c_conpa	PA194C, a cached-head 194-state machine. States that no machine below 194 was established.
40. pa194c_conpa_2026-11-13_1	PA194C. States a different 194-state machine and an article with a PA proof of the equivalence.
41. pa194c_conpa_2026-11-13	PA194C, a cached-head 194-state machine. States an article with a PA proof of the equivalence and nothing below 194.
42. pa194_radix_family_2026-11-13	New 194-state machines calibrated to Con(PA), a radix family.
43. pa194j_pa194d_conpa_2026-11-13	PA194J and PA194D. States two new 194-state machines.
44. pa194q_conpa_2026-11-13	PA194Q and PA194CSZ. States two new 194-state machines and that no machine below 194 was established.
45. PA194R_research_2026-11-14	Four 194-state machines R/F/A/X; zero-closing emitter, reassociated decoder, or source-entry known-zero decrement.
46. pa194N_conpa_2026-11-13	194-state N no-push successful-MP controller, with revised proof-to-command compiler.
47. pa194zr_conpa_2026-11-13	194-state ZR; shared comparator and emitter returns use known-zero register 3, nine source instructions and two rows.